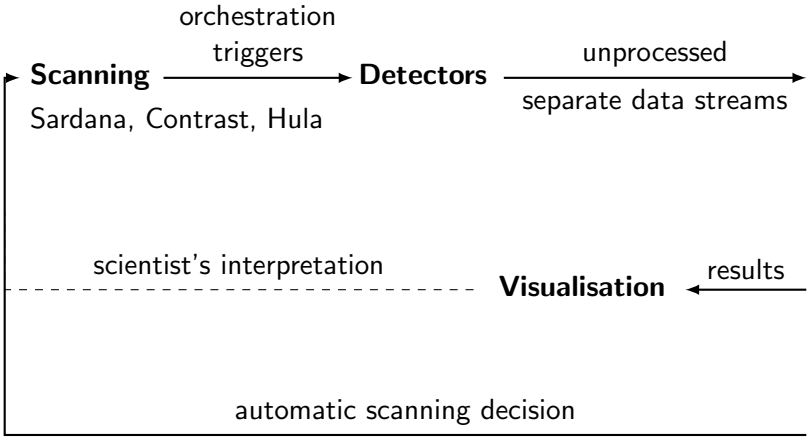
dranspo.se: Dynamic Live Processing Pipelines

Felix Engelmann, Scientific Data

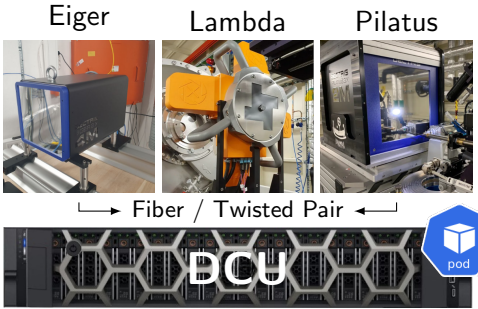
felix.engelmann@maxiv.lu.se

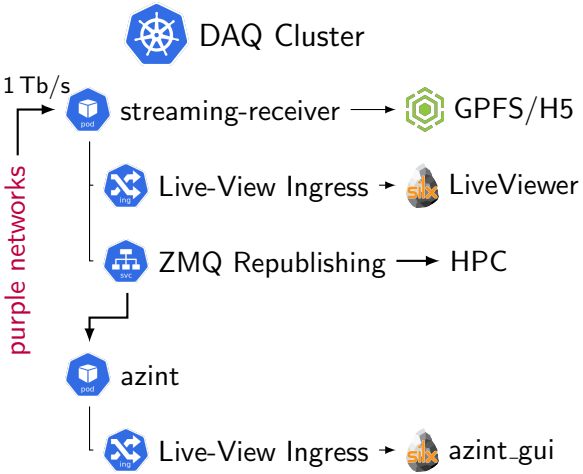
13th June 2024

Experimental Feedback Loop

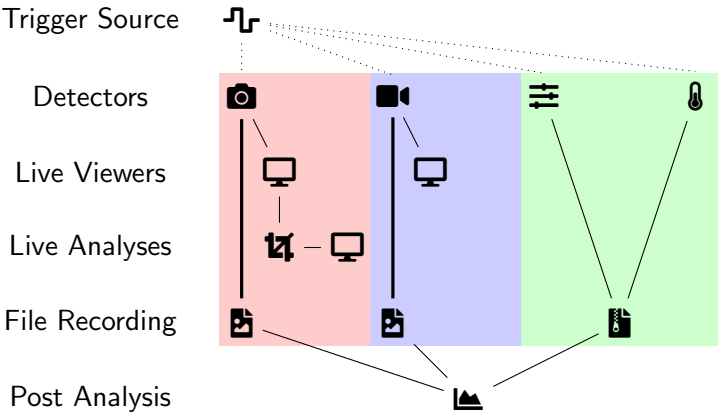


High-Bandwidth DAQ scheme





Existing Infrastructure



Limitations of Live Analysis

Single Data Stream

- only one detector data available
- simple tools, e.g. azint, crop, time integration
- beyond limits: normalisation to I_0, sorting by motor position

Custom Modules

- module development by SciDa
- custom deployment/integration
- custom live viewer interaction (mostly REST)

Additional Data Sources

Devices

PandAbox

XSpress3

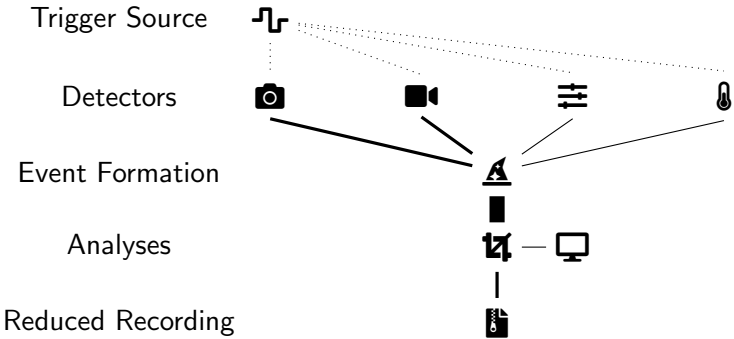
Basler

AlbaEM

Software

- Sardana
  - Icepap
  - Meta-data
  - Filename
  - Snapshots
- Contrast

Event Formation



Trigger Map

Event Definitions

Which *frames* from which *detectors* belong to the same *event* and have to be processed by the same *worker*?

Virtual Workers

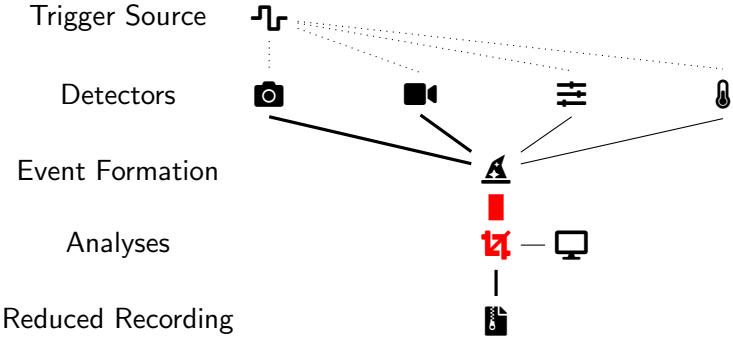
- virtual workers are dynamically assigned to real workers
- special *all* workers (stream headers), or discard frame with $\emptyset$
- *none* if stream has no frame for event

	all	1	3	5	7	8
	all	2	4	6	7	9
	all	all	none	none	$\emptyset$	none
	all	{1,2}	none	{5,6}	none	{8,9}

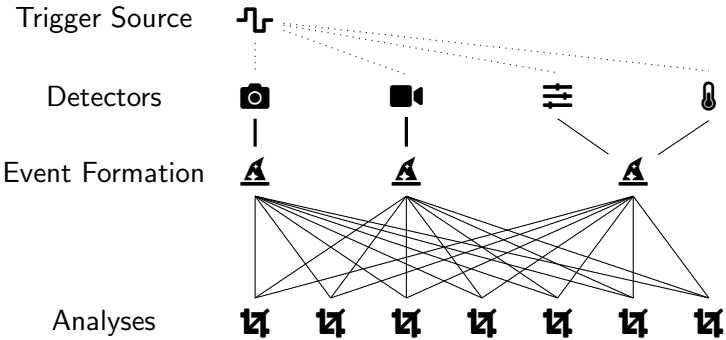
Scanning

trigger map specified by scanning software, append-only extendable

Event Formation Bottleneck



Parallelisation



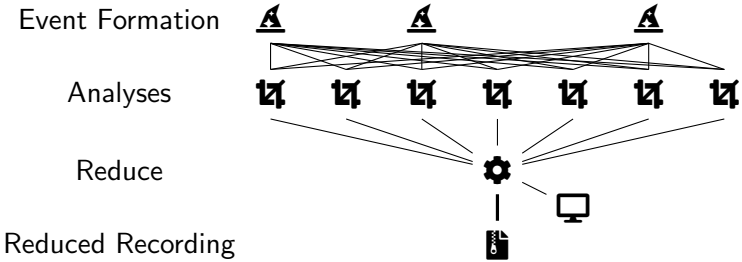
Missing Inter Event Analysis

- time integration
- temporal correlations

Sequential Reduce

Operations at Acquisition Speed

- append to list
- sum



Development: Testing

Recording

ingesters optionally write all streams to disk (sequential pickle dumps)

Replay

- from recorded zmq frames
- from hdf5 files

Local

run file-based ingesters, workers and reducer locally

Case Study: NanoMAX

Sources

- XSpress3
- Contrast (Motor Positions)

Analysis / Output

- PyMCA fluorescence fitting
- Concentration Map

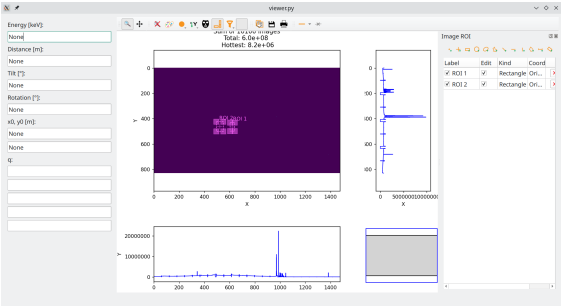
Case Study: FemtoMAX

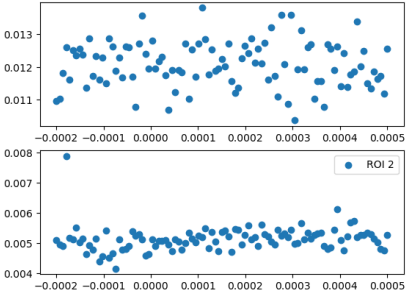
Sources

- Balor
- Zyla
- Sardana (x-ray delay)
- Pilatus

Analysis / Output

- Pileup
- ROI mean
- CMOS Single Photon extraction





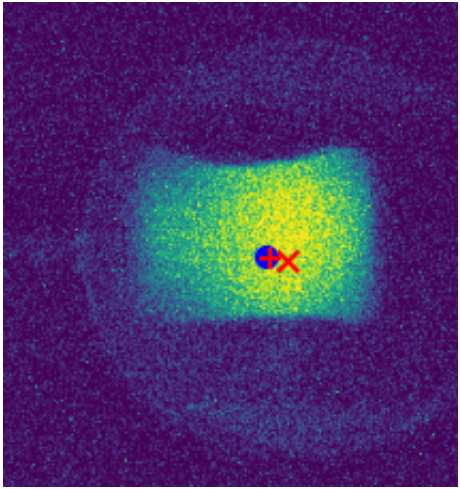
Case Study: DanMAX

Sources

- Basler
- Pilatus
- Orca
- PandAbox

Analysis / Output

- Beam CoG
- Slit Scan, Beam Alignment
- Pileup File
- Tomography Rotation Angle



```
pip install dranspose
```

Internals

Development: Worker

```python
class FluorescenceWorker:
    def __init__(self):
        self.number = 0

    def process_event(self, event: EventData,
                      parameters=None):
        print(event)
        # parse zmq frames
        # fit spectra to get concentrations
        # extract motor position
        return {"position": mot, "concentrations": ...}
```

reinstantiated for every scan (new Trigger Map)

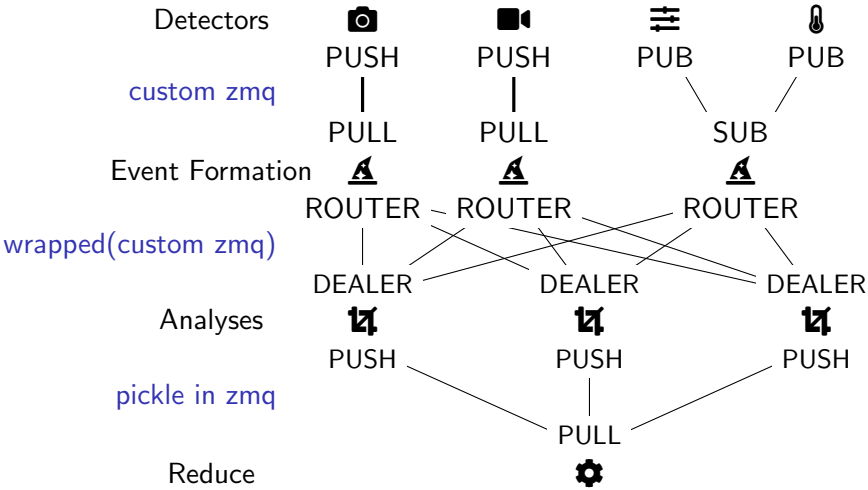
Development: Reducer

```
class FluorescenceReducer:
    def __init__(self):
        self.publish = {"map": {}}

    def process_result(self,
                       result: ResultData,
                       parameters=None):
        print(result.event_number)
        print(result.worker)
        print(result.parameters_uuid)
        data = result.payload
        self.publish["map"][data["position"]] = \
                data["concentrations"]
```

reinstantiated for every scan (new Trigger Map)

Architecture, ZMQ



Architecture, redis

config

- components publish config (connected peers, trigger map version, zmq url)
- timeout for liveness probe

updates

- controller notifies of new mapping/parameters

ready

- workers notify readyness after event processed

assign

event_number: EventNumber

assignments: dict[StreamName, list[WorkerName]]

Event Coordination

Controller

- wait for new entry in 🗄 ⏩ ready
- assign worker to first unassigned virtual worker (and *all*)
- distribute WorkAssignment in 🗄 ⏩ assign

Ingester

- filter assignment for own streams
- combine all local streams
- copy event to specified workers (ROUTER)

Worker

- filter assignment for own work
- listen to ingesters with participating streams
- assemble EventData
- call custom code
- send pickled result to reducer
- send ready message to 🗄 ⏩ ready

Common Modules

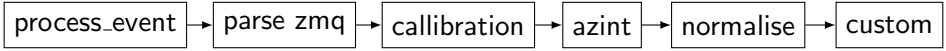
ZMQ Format / STINS

- unpacking of (mulit-part) zmq frames to numpy arrays

Calibration

- installed as python modules

Middlewares



- maybe register required parameters?
- registered in Worker `__init__`

Tests

End-to-End

- stream fake zmq frames
- full scan test

Protocols

- Pydantic BaseModel (similar to dataclass)
- all messages defined and validated
- no `dicts` with random fields

Typing

- type hint annotations
- `mypy` strict

Deployment

Docker

- install custom dependencies
- end-to-end build latency multiple minutes

K8s

- HELM chart for beamline
- restart pulls new version
- different containers for different experiments

Versioning

- add git commit hash to reduced data
- add parameters to h5 file

Performance

Bandwidth

- 10 Gbit/s from b-daq-cn2 and b-daq-cn3
- 8 workers

horizontally scalable if each stream $\leq$ 30 Gbit/s

Latency

- $\approx$ 2 kHz with enough workers

practically limited to $\approx n$ workers $\Rightarrow$ max worker runtime $\frac{n}{\text{acquisition rate [Hz]}}$s

Virtual Worker Distribution

$\forall \text{st}_0 \in \mathbb{N}, h \in [|\text{workers}|, \infty)$:

$$|\{M_{\text{ev,st}} : \text{st}_0 \leq \text{ev} < (\text{st}_0 + h), \text{st} \in \text{streams}\}| > h - \epsilon$$

Outlook

File Writing

- custom by developers?

Autoscaling

- observability of workers (queues)
- duty cycle of workers
- non-deterministic worker functions
- integration with k8s

Scan Integration

- publish trigger map
- append to trigger map